K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Name of Student : Harshal Nitin Mehta
Roll No : B - 63
Batch : SB4

EXPERIMENT 1

Aim :- Implementation of Stack using Array for real-world application.

Objectives :-

1. To understand the concepts of **data structures and analysis procedures**.
2. To understand **linear data structures** and their implementation for various real-world applications.

Theory :-

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. It allows insertion and deletion of elements from only one end, called the **TOP**.

The main operations performed on a stack are:

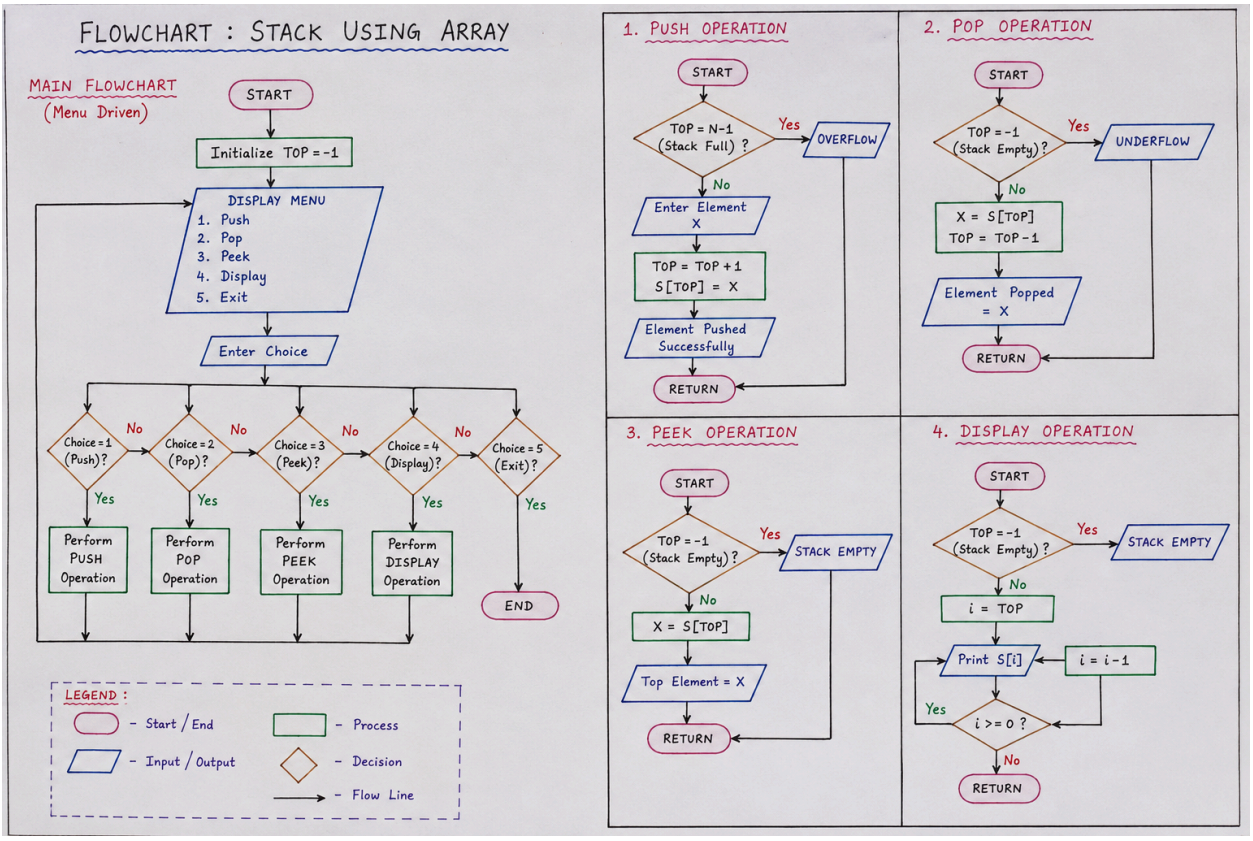
- **Push** – Inserts an element at the top of the stack.
- **Pop** – Removes the topmost element from the stack.
- **Peek** – Displays the top element without removing it.

- **Display** – Displays all elements present in the stack.

Stacks can be implemented using **arrays or linked lists**. They are widely used in applications such as **function calls, expression evaluation, recursion, and undo/redo operations**.

Operation	Description
Push	Inserts a new element at the top of the stack if the stack is not full.
Pop	Removes the topmost element from the stack if the stack is not empty.
Peek	Displays the top element without removing it from the stack.
Display	Shows all the elements present in the stack from top to bottom.

These operations help in efficiently managing data according to the **LIFO principle**.



PROGRAM :-

Conditions-

Temperature>=40 deg print(Yellow Alert)

Temperature>=43 deg print(Orange Alert)

Temperature>=45 deg print(Red Alert)

Temperature<40 deg: remove all active alerts, Pop.

```
#include <stdio.h>

int stack[100], top = -1;

void Push(int x)

{
```

```
    stack[++top] = x;
}
void Pop()
{
    if (top == -1)
        printf("\nNo active alerts.");
    else {
        printf("\nPopped: %s",
                stack[top] == 1 ? "Yellow Alert" :
                stack[top] == 2 ? "Orange Alert" : "Red Alert");
        top--;
    }
}
void Peek()
{
    if (top == -1)
        printf("\nNo active alerts.");
    else
        printf("\nTop Alert: %s",
                stack[top] == 1 ? "Yellow Alert" :
                stack[top] == 2 ? "Orange Alert" : "Red Alert");
}
```

```
void Display()
{
    int i;
    if (top == -1)
        printf("\nNo active alerts.");
    else
        for (i = top; i >= 0; i--)
            printf("\n%s",
                stack[i] == 1 ? "Yellow Alert" :
                stack[i] == 2 ? "Orange Alert" : "Red Alert");
}
int main()
{
    int ch, t;
    do {
        printf("\n\n--- HEATWAVE ALERT SYSTEM ---");
        printf("\n1. Enter Temperature");
        printf("\n2. Push Alert");
        printf("\n3. Pop Alert");
        printf("\n4. Peek Alert");
        printf("\n5. Display Alerts");
        printf("\n6. Exit");
        printf("\nChoice: ");
```

```
scanf("%d", &ch);

if (ch == 1) {
    printf("Temperature: ");
    scanf("%d", &t);

    if (t >= 45) {
        printf("Red Alert");
        Push(3);
    }
    else if (t >= 43) {
        printf("Orange Alert");
        Push(2);
    }
    else if (t >= 40) {
        printf("Yellow Alert");
        Push(1);
    }
    else {
        printf("Temperature < 40");
        while (top != -1)
            Pop();
    }
```

```
        }
        else if (ch == 2) {
            printf("\nEnter alert (1-Yellow, 2-Orange, 3-Red): ");
            scanf("%d", &t);
            Push(t);
            printf("Alert Pushed.");
        }
        else if (ch == 3)
            Pop();
        else if (ch == 4)
            Peek();
        else if (ch == 5)
            Display();

        else if (ch == 6)
            printf("\nProgram Finished.");
        else
            printf("\nInvalid Choice.");
    } while (ch != 6);
    return 0;
}
```

OUTPUT :-

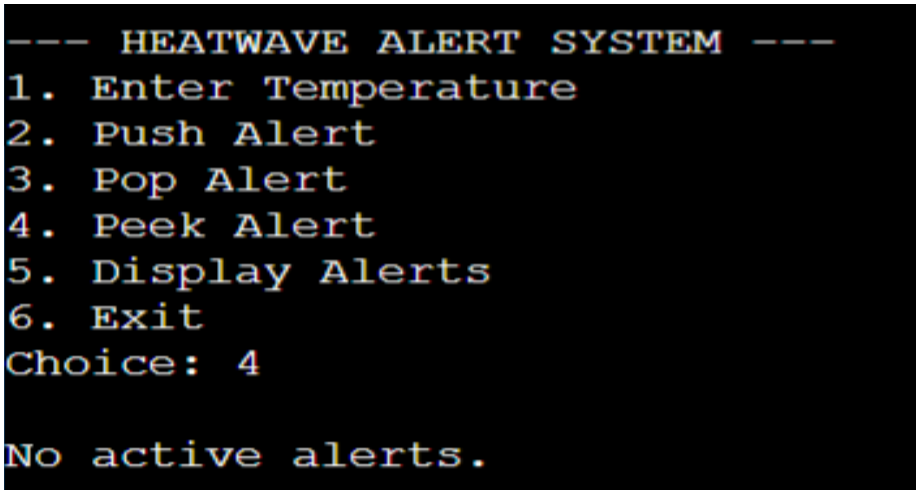
```
--- HEATWAVE ALERT SYSTEM ---
1. Enter Temperature
2. Push Alert
3. Pop Alert
4. Peek Alert
5. Display Alerts
6. Exit
Choice: 1
Temperature: 10
Temperature < 40
```

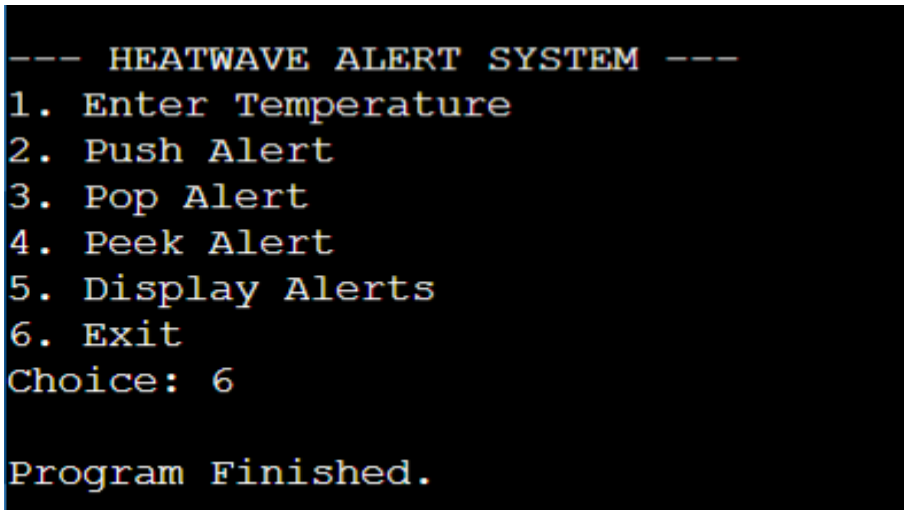
```
--- HEATWAVE ALERT SYSTEM ---
1. Enter Temperature
2. Push Alert
3. Pop Alert
4. Peek Alert
5. Display Alerts
6. Exit
Choice: 2

Enter alert (1-Yellow, 2-Orange, 3-Red): 2
Alert Pushed.
```

```
--- HEATWAVE ALERT SYSTEM ---
1. Enter Temperature
2. Push Alert
3. Pop Alert
4. Peek Alert
5. Display Alerts
6. Exit
Choice: 3

Popped: Orange Alert
```





Outcome :-

After completing this experiment, I can implement a stack using an array in C. I am able to perform basic stack operations and understand how the **LIFO principle** works in practice.

I can also identify **stack overflow and underflow conditions** while writing programs. I learned how stacks are used in real-world applications such as **expression evaluation, recursion, and undo/redo features**.

Conclusion :-

Through this experiment, I understood the concept and working of the stack data structure using an array. I learned how **Push, Pop, Peek, and Display** operations are implemented and how they follow the **LIFO principle**.

I also understood the conditions of **stack overflow and stack underflow**. Overall, the experiment gave me a clear understanding of stack implementation and its importance in real-world applications.

Algorithms :-

1. Stack Push

Step 1: Check Overflow

If TOP = N, then write **"Overflow"** and return.

Step 2: Increment TOP

TOP ← TOP + 1

Step 3: Insert Element

S[TOP] ← Y

Step 4: Return

---

2. Stack Pop

Step 1: Check Underflow

If TOP = 0, then write **"Underflow"** and return.

Step 2: Delete Element

Y ← S[TOP]

Step 3: Decrement TOP

TOP ← TOP - 1

Step 4: Return Element

Return(Y)

---

3. Stack Peep

Step 1: Check Valid Position

If TOP - POS + 1 ≤ 0, then write **"Invalid Position"** and return 0.

Step 2: Find Element

Y ← S[TOP - POS + 1]

Step 3: Return Element

Return(Y)

---

4. Stack Display

Step 1: Check Underflow

If TOP = 0, then write **"STACK EMPTY"** and return.

Step 2: Display Elements

For i ← TOP down to 1:

Write(S[i])

Step 3: Return
